GATE
CS & DA

Artificial Intelligence

Un- Informed Search

Lecture No. 1

By- SIDDHARTH SABHARWAL SIR

Recap of Previous Lecture

- Topic NIL
- Topic
- Topic
- Topic
- Topic

Topics to be Covered

- Topic
- Topic
- Topic
- Topic
- Topic

Uninformed Search

BFS

(No prerequisite)

Syllabus

Search: informed, uninformed, adversarial; logic, propositional, predicate; reasoning under uncertainty topics - conditional independence representation, exact inference through variable elimination, and approximate inference through sampling.

⇒ (2024: 7 marks)

(DPP/Class notes will be enoughs.)

not Imp.

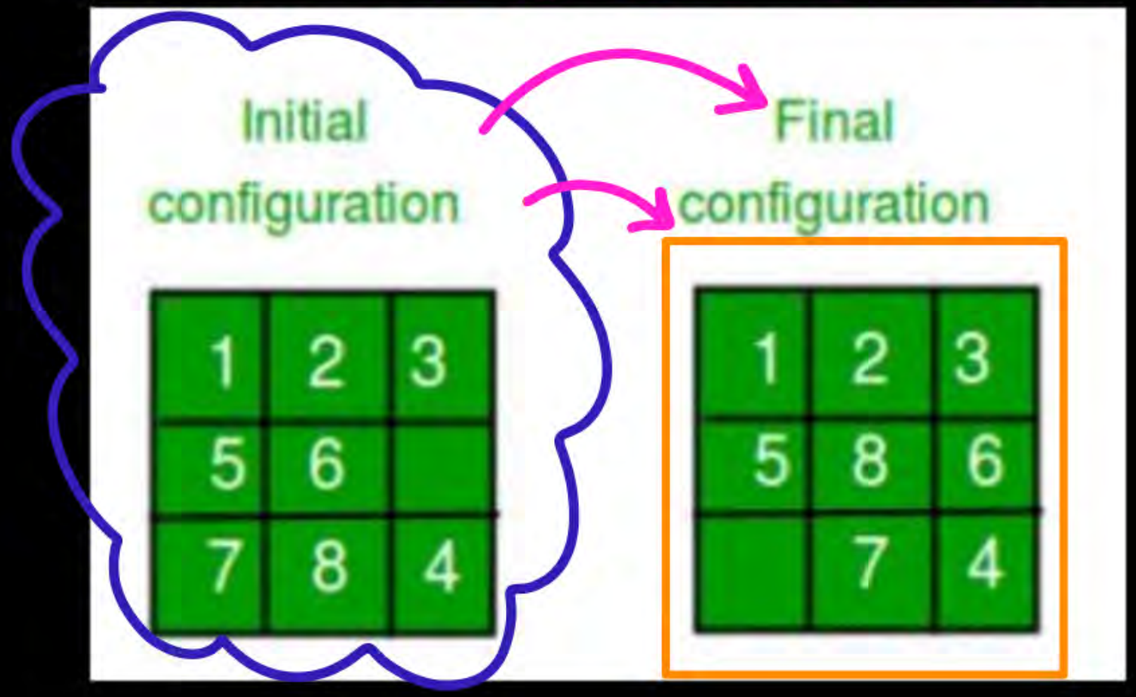
What is Artificial Intelligence

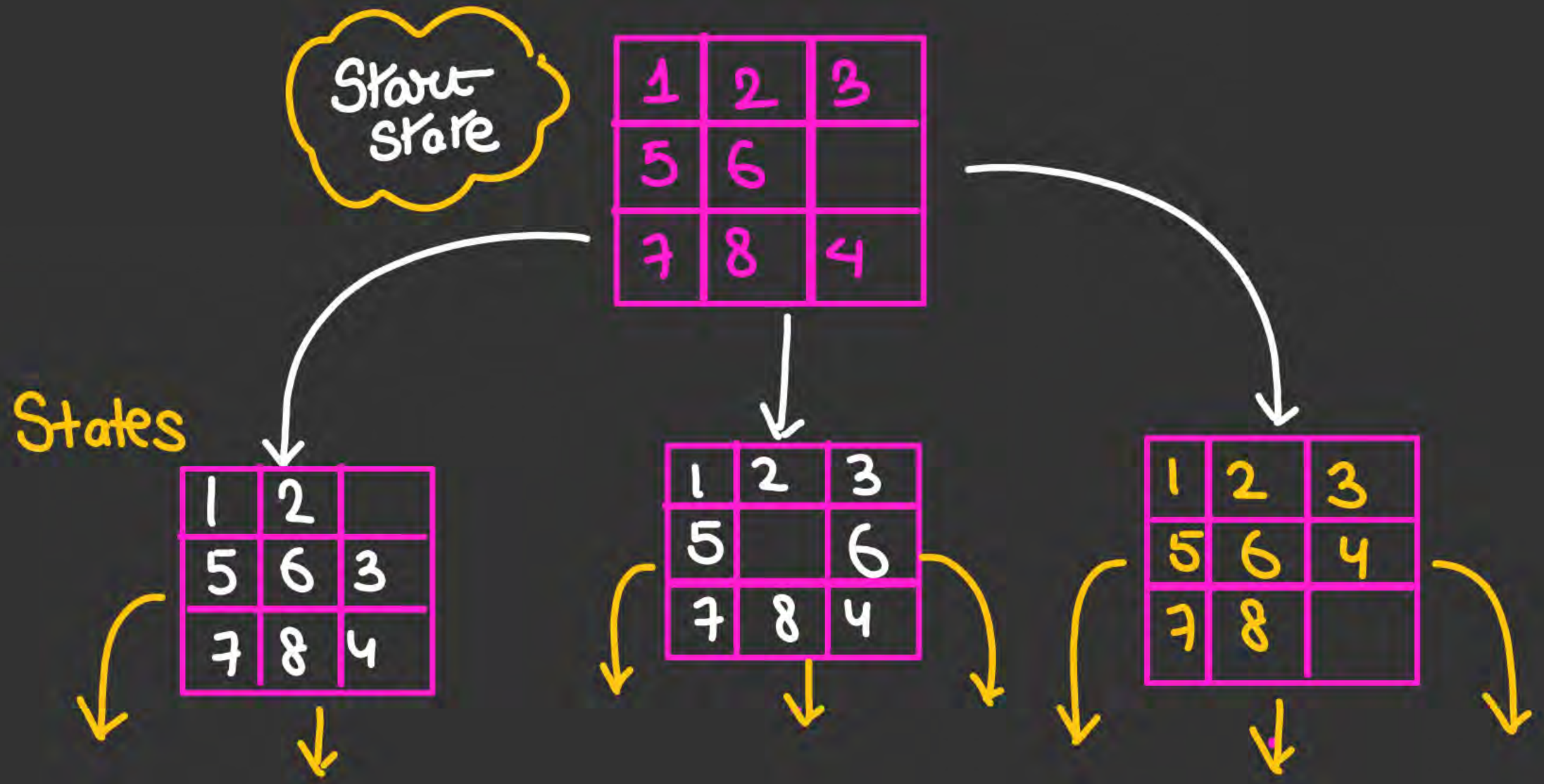
- Artificial Intelligence (AI) is a branch of computer science that focuses on creating machines and software capable of performing tasks that typically require human intelligence.
- AI can cause a machine to work as human.
- Artificial (man made) and Intelligence (power of thinking)
- These tasks include learning, reasoning, problem-solving, perception, language understanding, and interaction.

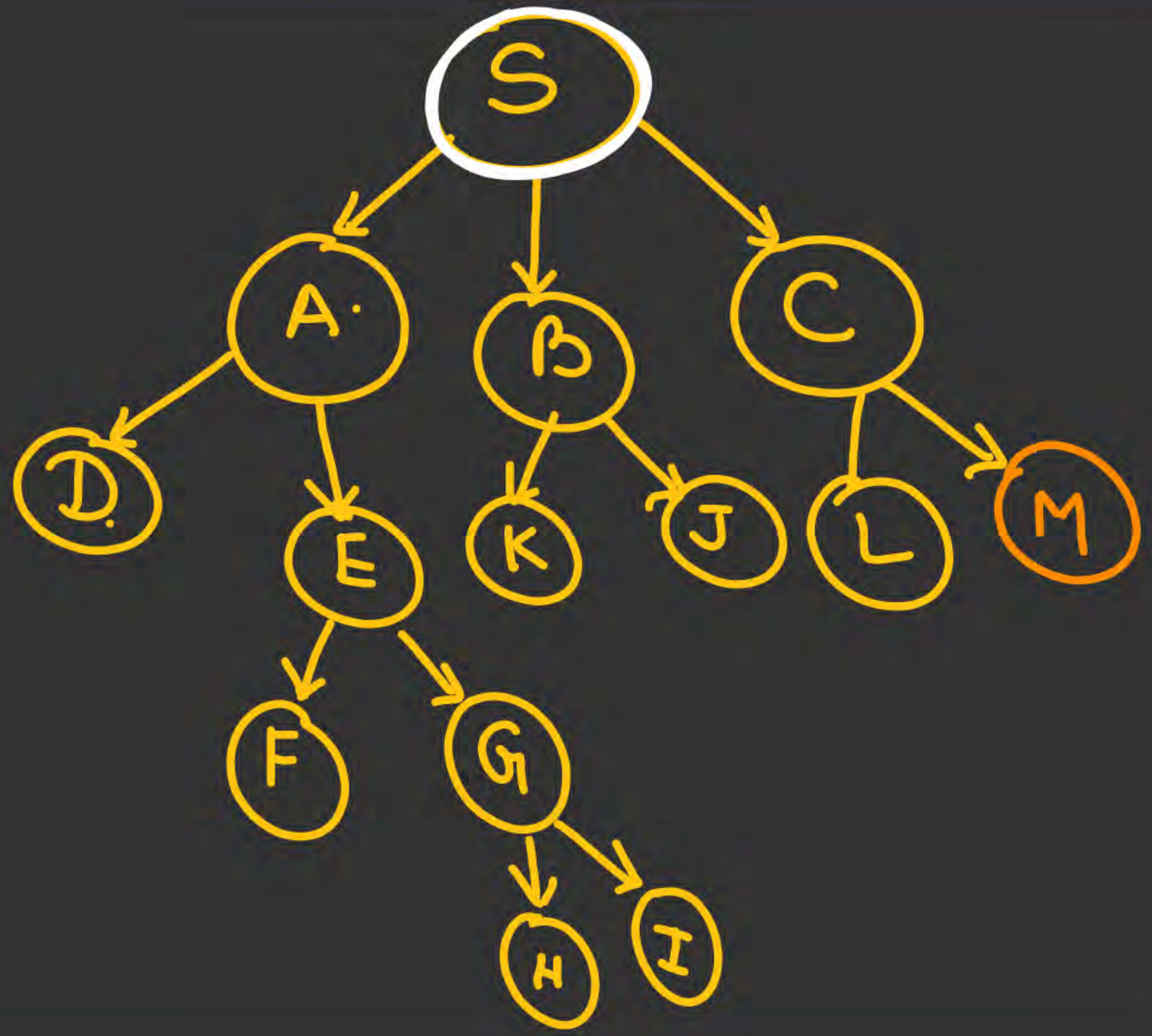
Reference

What is a State and State Space...

- Example Eight tile puzzle : Given a 3×3 board with 8 tiles (every tile has one number from 1 to 8) and one empty space. The objective is to place the numbers on tiles to match the final configuration using the empty space. We can slide four adjacent (left, right, above, and below) tiles into the empty space.




Start state
1 2 3
5 6
7 8 4
States
1 2
5 6 3
7 8 4
1 2 3
5 6
7 8 4
1 2 3
5 6 4
7 8



- These are States.
- State space.

- Starting state
- goal state
- Search??

only for reference.

What is a State and State Space...

- State: A specific configuration of the system at a particular point in time.
- State Space: The entire set of possible states that the system can be in, including the initial state, goal state(s), and all intermediate states.
- State Space Search: Used in Problem Solving. It is a process used in A.I in which Successive Configurations or States of an instance are considered with intention of a GOAL State with desired property.

we are moving from start state and search for goal state in state space

What is a State and State Space...

- State space search is a fundamental concept in artificial intelligence (AI) that involves exploring possible states of a system to find a solution to a problem.
- An essential method in artificial intelligence is state space search, which looks for potential states and their transitions to solve issues.
- According to this method, the problem is modeled as a state space, with each state representing a possible configuration and transitions denoting actions or operations that change the state of the problem. Finding a route that meets predetermined requirements from an initial state to a goal state is the aim.

What is a State and State Space...

- Lets try to solve this :

What is a State and State Space...

- Initial State: The starting point of the search.
- Goal State(s): The desired state(s) that signify a solution.
- Actions/Operators: The set of possible actions that can be performed to move from one state to another.
- Transition Model: Describes the result of applying an action to a state, leading to a new state.
- Path Cost: A function that assigns a cost to each path, helping to find the most efficient solution.

Types of Search in AI

- In AI, search algorithms can be broadly categorized into three types:
  - uninformed (blind) search,
  - informed (heuristic) search,
  - and adversarial search.

Uniformed search.

- no info abt path
- Only goalstate$
  Start. State
- we try to follow all path until goalstate is Reached.

How we follow these paths is defined by the algorithm we use in un-informed search.

Uninformed Search

- Uninformed search is a class of general-purpose search algorithms which operates in brute force-way. Uninformed search algorithms do not have additional information about state or search space other than how to traverse the tree, so it is also called blind search.
- The search algorithms in this section have no additional information on the goal node other than the one provided in the problem definition. The plans to reach the goal state from the start state differ only by the order and/or length of actions. Uninformed search is also called Blind search. These algorithms can only generate the successors and differentiate between the goal state and non goal state.

Uninformed Search

- Uninformed search algorithms do not have any additional information about the goal state other than the information provided in the problem definition. They explore the search space blindly without any guidance or heuristics to indicate which path is likely to lead to the goal.
- The following uninformed search algorithms are discussed in this section.
- Depth First Search
- Breadth First Search
- Uniform Cost Search

Uninformed Search

- Each of these algorithms will have:
- A problem graph, containing the start node S and the goal node G.
- A strategy, describing the manner in which the graph will be traversed to get to G.
- Store all the possible states (nodes) that you can go from the current states.
- A solution plan, which the sequence of nodes from S to G.

Question.

Breadth first Search

- Breadth-first search (BFS) is an algorithm for traversing or searching tree or graph data structures.
- It starts at the tree root (or some arbitrary node of a graph, sometimes referred to as a 'search key'), and explores all of the neighbor nodes at the present depth prior to moving on to the nodes at the next depth level.
- It is implemented using a queue (FIFO).
- Breadth First Search (BFS) is a graph traversal algorithm that explores all the vertices in a graph at the current depth before moving on to the vertices at the next depth level. It starts at a specified vertex and visits all its neighbors before moving on to the next level of neighbors. BFS is commonly used in algorithms for pathfinding, connected components, and shortest path problems in graphs.

Breadth first Search

- The Breadth-First Search is a traversing algorithm used to satisfy a given property by searching the tree or graph data structure.
- It belongs to uninformed or blind search AI algorithms as It operates solely based on the connectivity of nodes and doesn't prioritize any particular path over another based on heuristic knowledge or domain-specific information.
- it doesn't incorporate any additional information beyond the structure of the search space. It is optimal for unweighted graphs and is particularly suitable when all actions have the same cost. Due to its systematic search strategy, BFS can efficiently explore even infinite state spaces.

Breadth first Search

- Breadth-First Search (BFS) is an algorithm for traversing or searching tree or graph data structures.
- In artificial intelligence (AI), BFS is commonly used for finding the shortest path in **unweighted graphs,** exploring state spaces in search problems, and ensuring all nodes at a given depth are processed before moving deeper.

Breadth first Search

Key Characteristics

- Traversal Method: BFS explores all nodes at the present depth level before moving on to nodes at the next depth level.
- FIFO Structure: Uses a queue (First-In-First-Out) data structure.
- Completeness: BFS is complete, meaning it is guaranteed to find a solution if one exists.

Breadth first Search (Example)

- BFS ⇒ S ⟶ G, we create 2 List

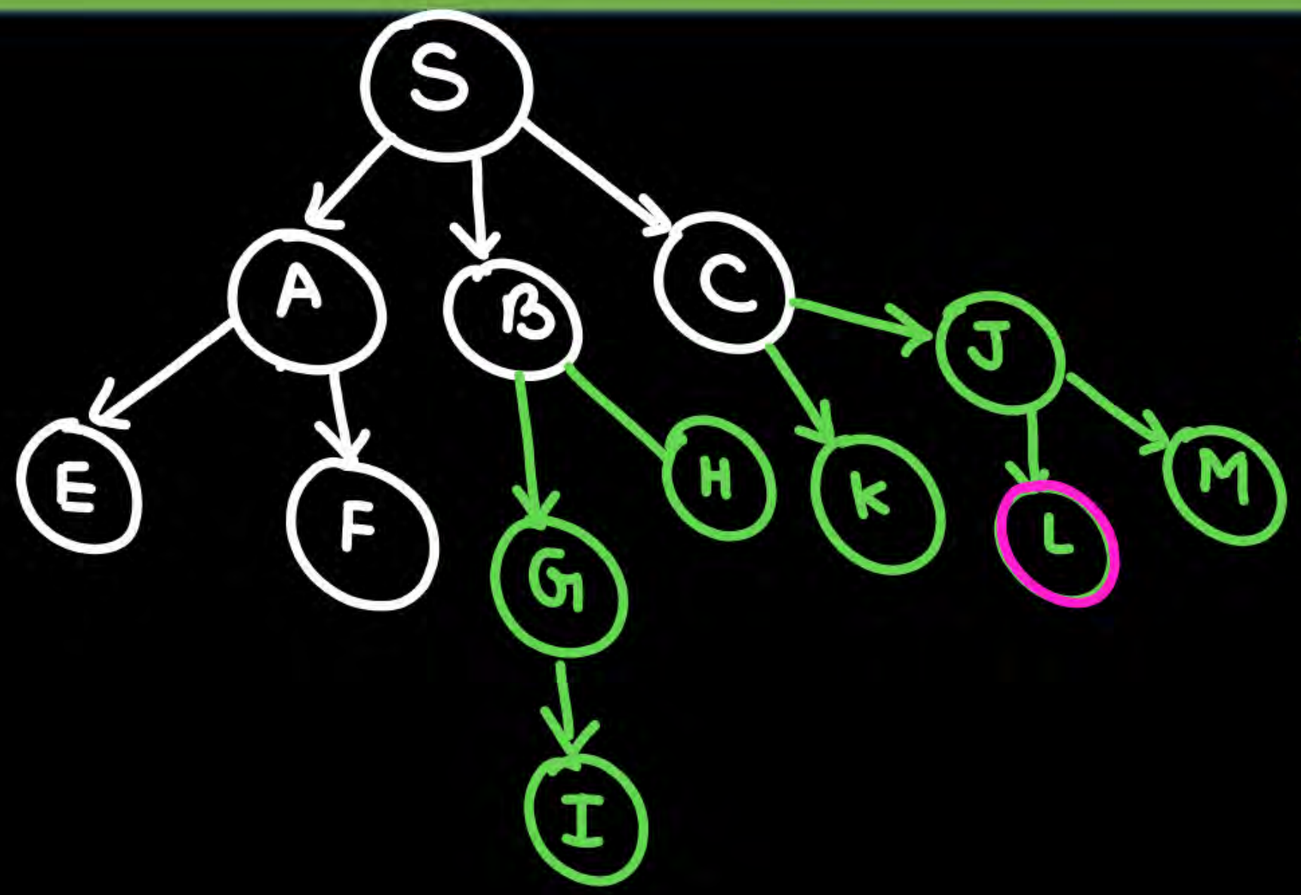
Visited Nodes

Queue : FIFO

exist ← Queue : FIFO ← enter

- Step1: we start with the Start node. enter the Start node into Q.
- Step 2: Now Remove the first node in Q and. insert it's children (immediate) from (L to R) into the Q. (delete node from left of Q and insert from Right)
- Step 3 Repeat step2 until goal node is Reach.

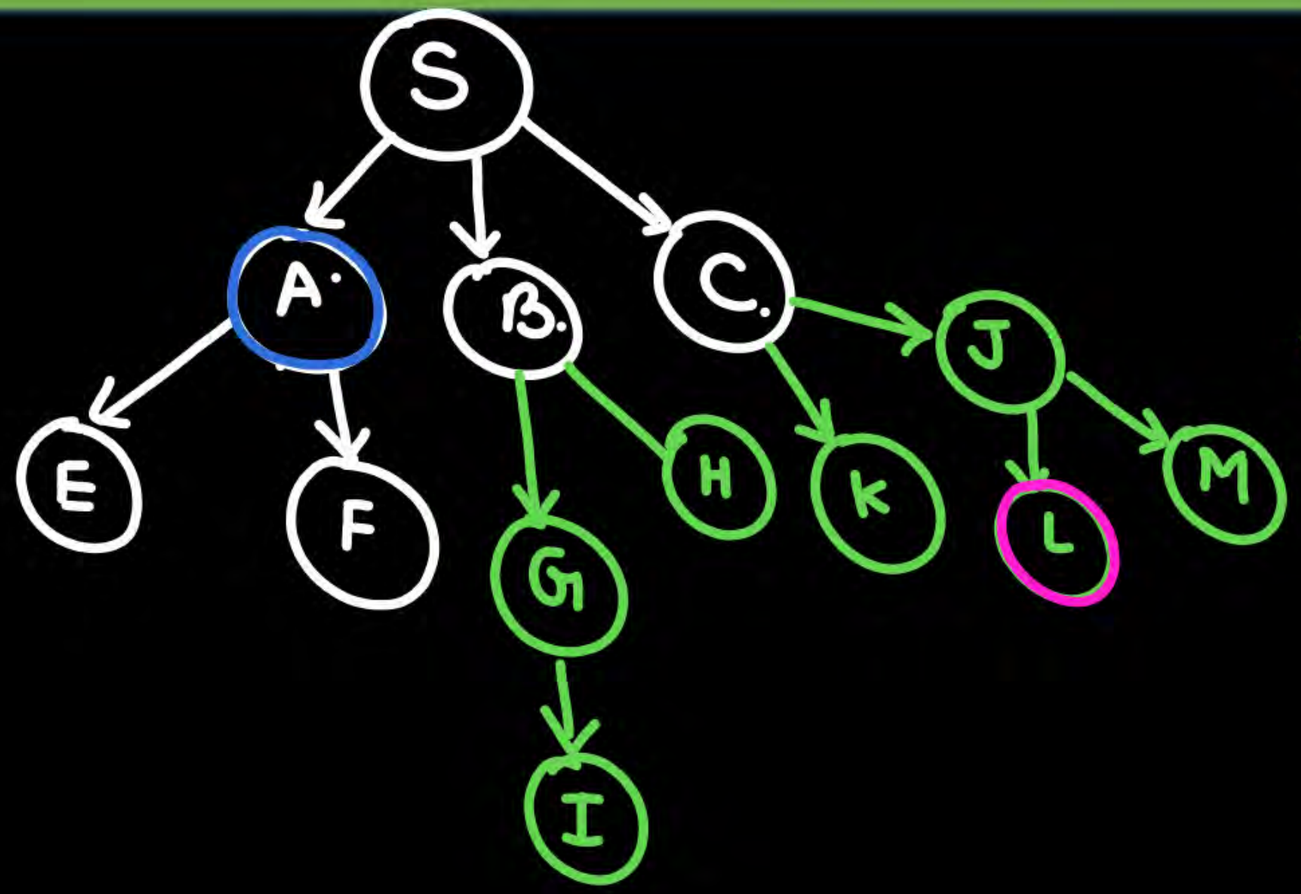
Breadth first Search (Example)



V S

Q S

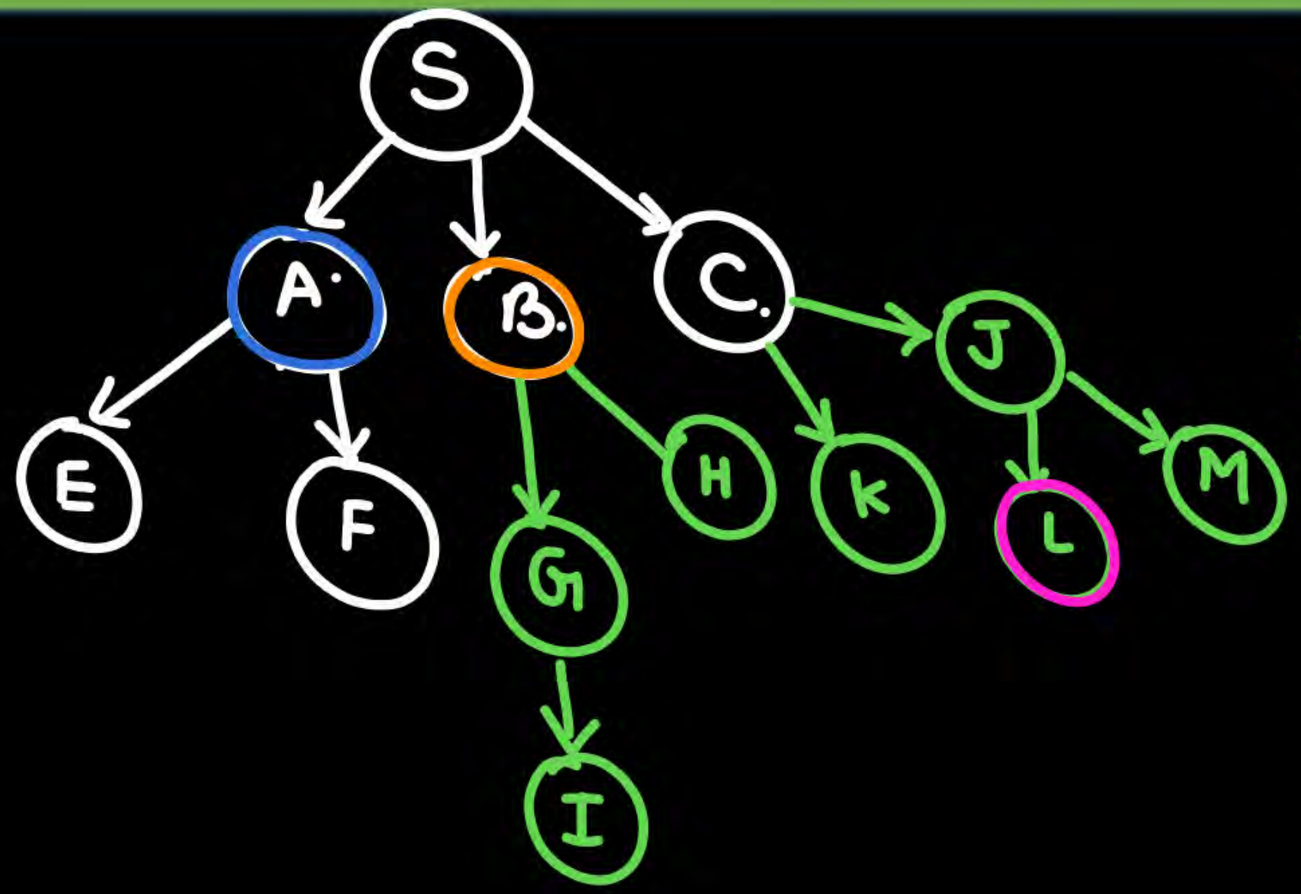
Breadth first Search (Example)



V SA

Q ~~S~~ ~~A~~BC

Breadth first Search (Example)



V SAB

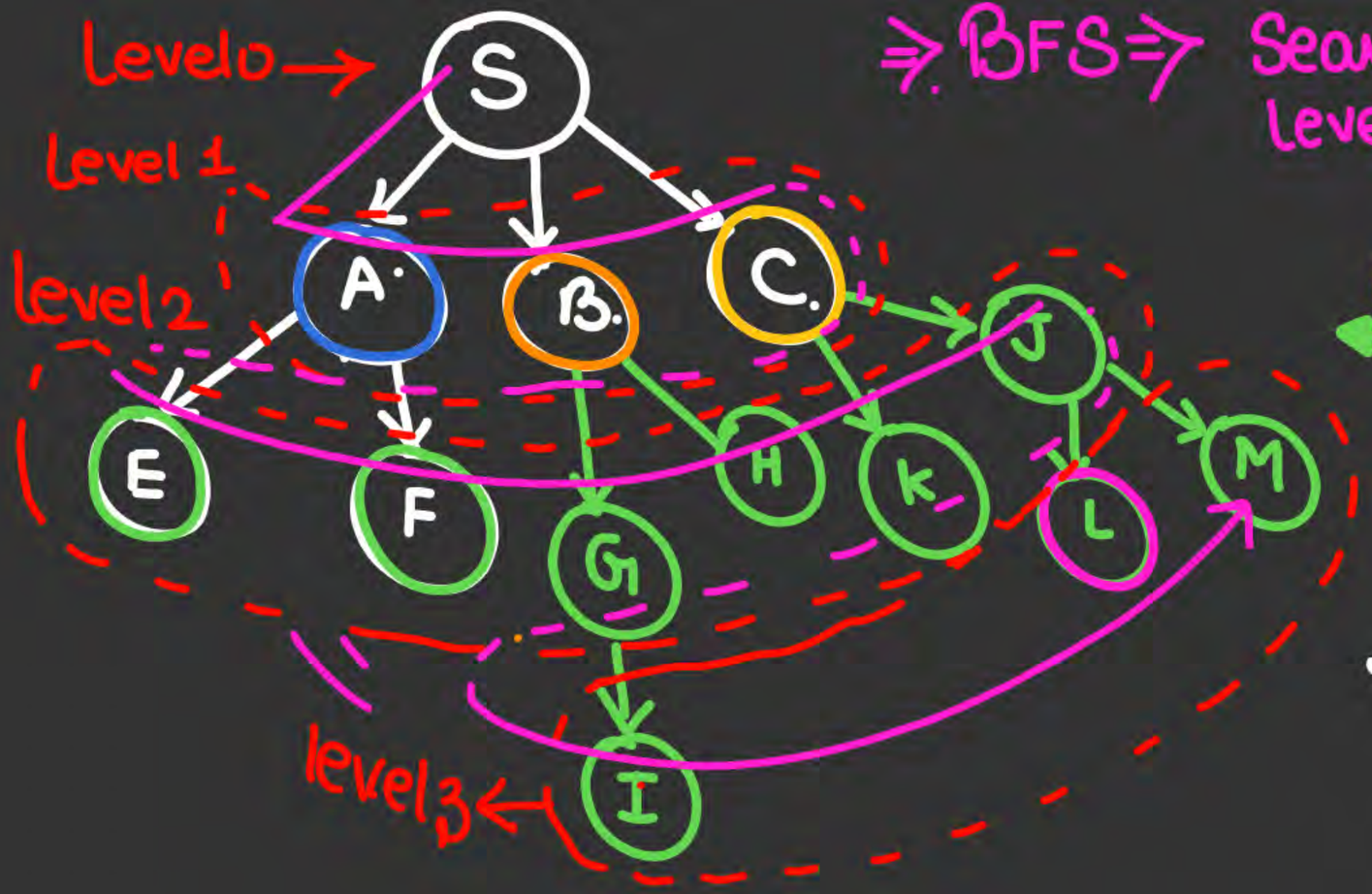
Q ~~S~~ ~~A~~ ~~B~~ CEFGH

How we visited nodes

V SABCEFGHKJILM

⇒ BFS ⇒ Search goal node level by level

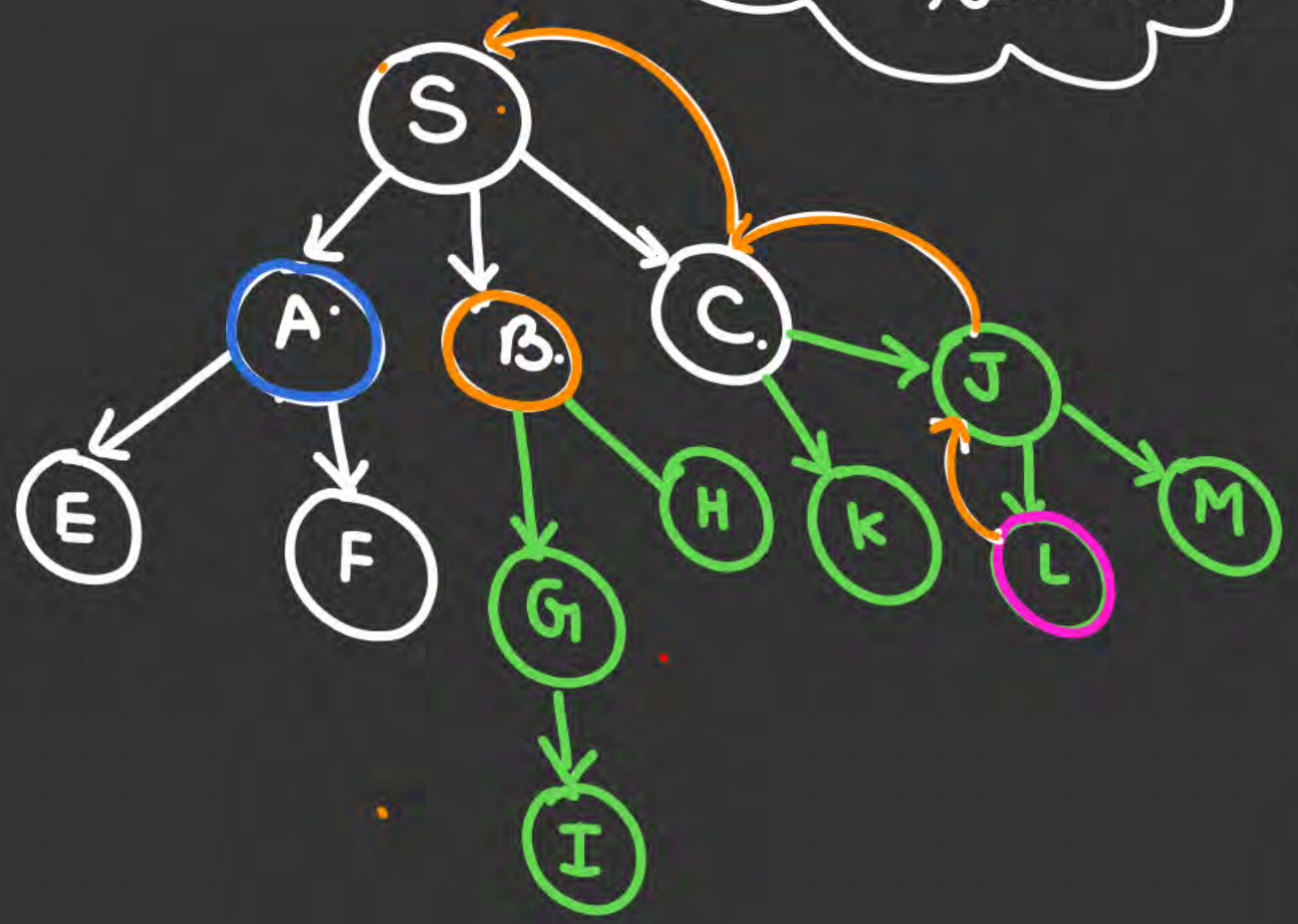
So BFS travel from all nodes.



Q ~~S~~ ~~A~~ ~~B~~ ~~C~~ ~~E~~ ~~F~~ ~~G~~ ~~H~~ ~~K~~ ~~J~~ ~~I~~ ~~L~~ ~~M~~

- So goal node can be searched easily.

Parent to Parent



S A B C E F G H K J

I L | Stop as goal state reached

Q

~~S A B C E F G H K J~~

~~I~~ L m

Path is Parent to Parent

SCJL

One more example - here one maze ,

S 0 0 0

0 1 1 0

0 1 0 G

0 0 0 0

In this maze:

- S: Starting point
- G: Goal
- 0: An open path
- 1: An obstacle (you can't pass through it)

Breadth first Search

Algorithm Initialization: Start at the root node (or any arbitrary node in a graph). We create 2 list (visited and Queue)

- Mark the node as visited.
- Enqueue the node.
- Process: While the queue is not empty
- Dequeue a node from the queue.
- Process the node (e.g., check if it is the goal).
- For each unvisited adjacent node
- Mark it as visited
- Enqueue it.

Breadth first Search

Advantages

- Optimal for Unweighted Graphs: BFS will find the shortest path in terms of the number of edges.
- Complete: Guaranteed to find a solution if one exists.
- Simple Implementation: Straightforward to implement using a queue.

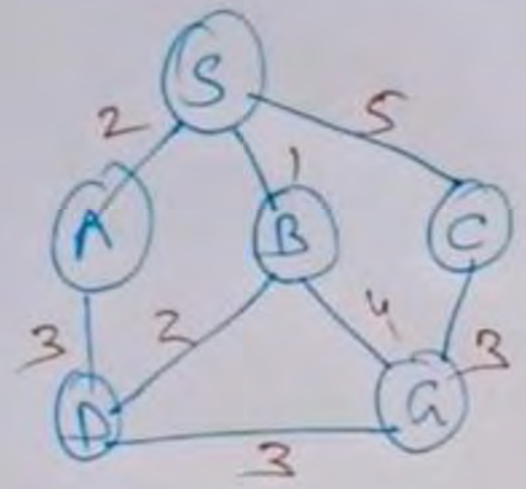
Breadth first Search

Disadvantages

- Memory Usage: Can consume a lot of memory, as it needs to store all nodes at the current depth level.
- Not Suitable for Weighted Graphs: BFS does not account for edge weights; Dijkstra's algorithm is more appropriate in such cases.

Breadth first Search

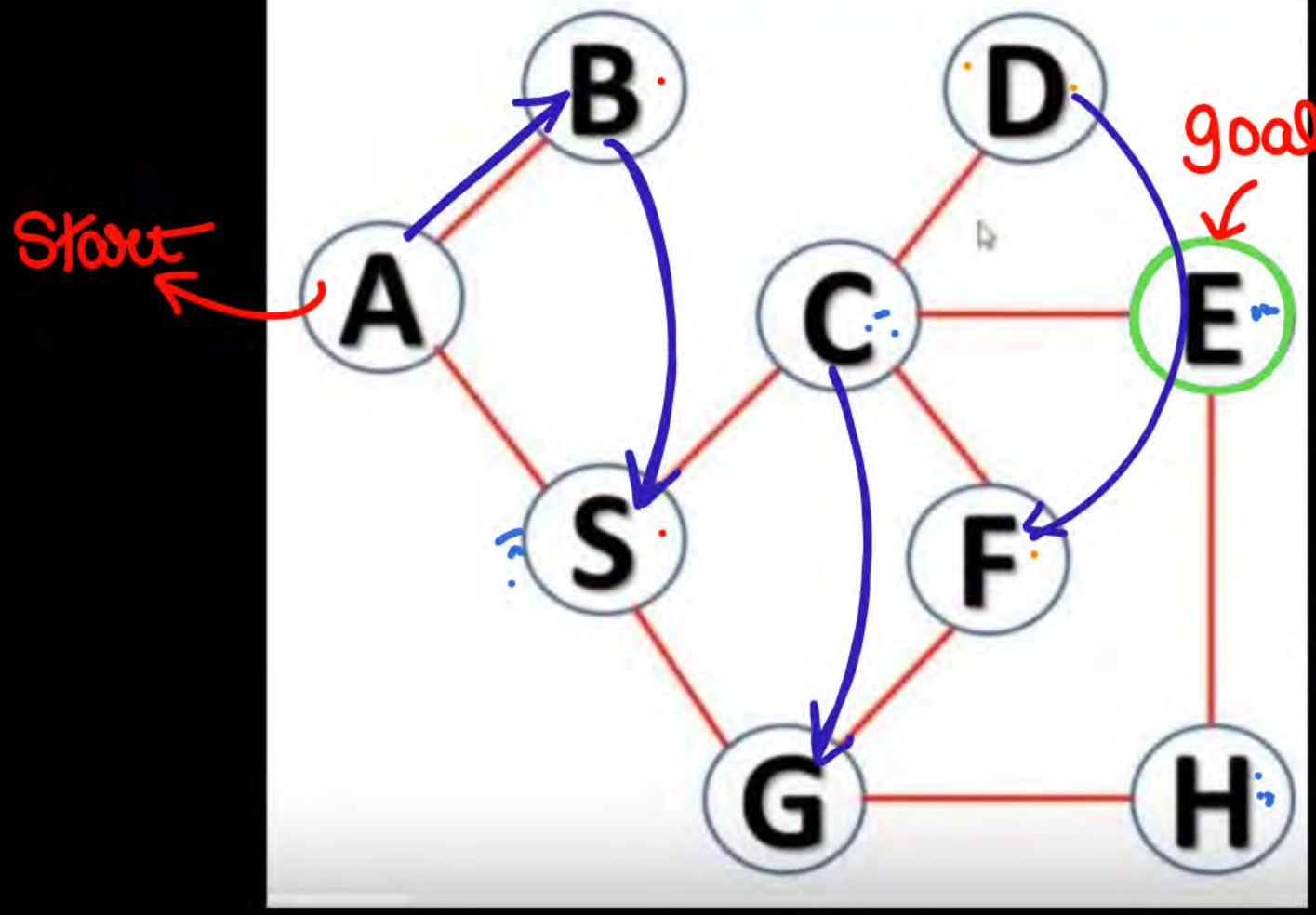
- This algorithm always give a result (if it exists)
- Because before removing the node form the queue we check it with goal node
- And from root to the desired node we can easily follow the parent – parent and get the path from root to the desired node.
- Path Reconstruction : To find the path from A to G, we backtrack using the parent information:



find the route from S to G using BFS

Get enter nodes in alphabetical order

BREADTH FIRST SEARCH



Visited

A B S C G D E | stop

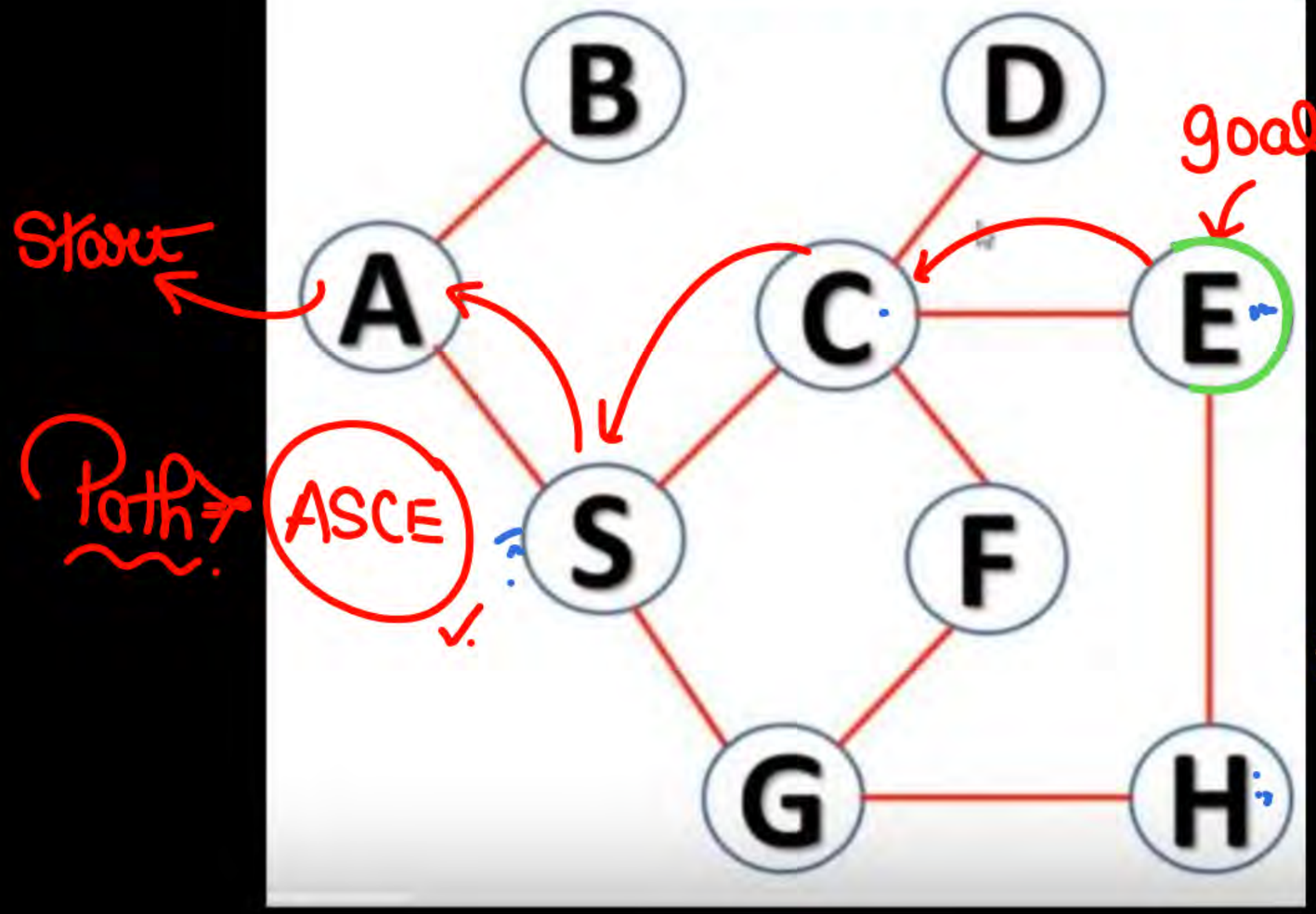
Queue

~~A~~ ~~B~~ ~~S~~ ~~C~~ ~~G~~ ~~D~~ ~~E~~ F H

*So if any node or state is present in any list no need to re enter it

Get enter nodes in alphabetical order

BREADTH FIRST SEARCH



Visited
ABSCGDE | Stop.

Queue
~~A~~ ~~B~~ ~~S~~ ~~C~~ ~~G~~ ~~D~~ ~~E~~ F H

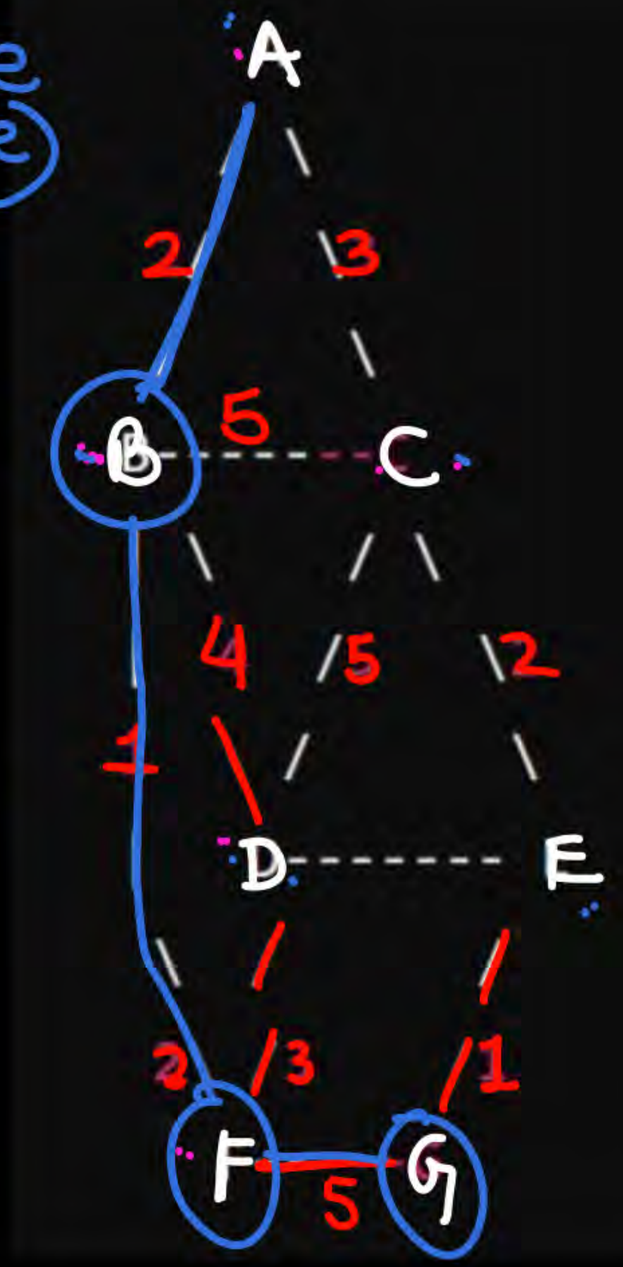
*So if any node or state is present in any list no need to re enter it

- A to G using BFS ⇒ (weights are waste)

[alphabetical order]

Visited: A B C D F E G

Queue: ~~A~~ ~~B~~ ~~C~~ D ~~F~~ ~~E~~ ~~G~~



Breadth first Search

- This always give the shortest path or the optimal results always.
- Here we don't have weights on the links

Breadth first Search

- BFS is useful for analyzing the nodes in a graph and constructing the shortest path of traversing through these.
- BFS can traverse through a graph in the smallest number of iterations.
- The architecture of the BFS algorithm is simple and robust.
- The result of the BFS algorithm holds a high level of accuracy in comparison to other algorithms.
- BFS iterations are seamless, and there is no possibility of this algorithm getting caught up in an infinite loop problem.

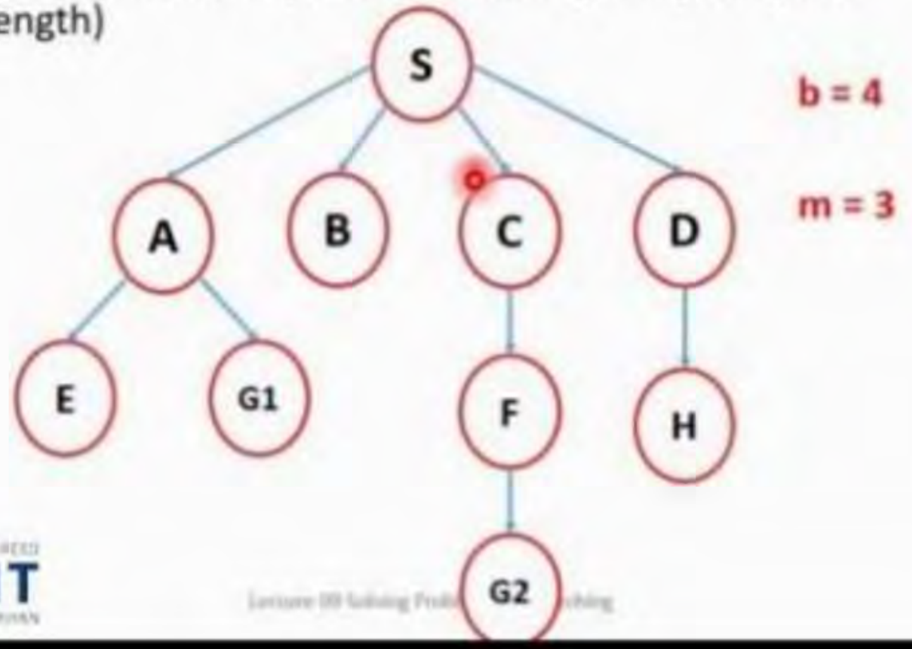
Time Complexity: Time Complexity of BFS algorithm can be obtained by the number of nodes traversed in BFS until the shallowest Node. Where the d= depth of shallowest solution and b is a node at every state.

$$T(b) = 1+b^2+b^3+\ldots\ldots+b^d = O(b^d)$$

Space Complexity: Space complexity of BFS algorithm is given by the Memory size of frontier which is $O(b^d)$.

Comparing Uninformed Search Strategies

- Time and space complexity are measured in
  - b – maximum branching factor of the search tree
  - m – maximum depth of the state space (Max path length)



Breadth first Search

- Time Complexity:
- In the worst case, BFS also has a time complexity of $O(B^D)$(Exponential).
- However, BFS systematically explores all nodes at each depth level before moving to the next level.
- It is particularly efficient in finding the shortest path in an unweighted graph because it
- guarantees that the shallowest goal node is reached first.

Breadth first Search

- Space Complexity:
- The space complexity of BFS is typically $O(B^D)$ (Exponential) because it stores all nodes bd at each depth level in a queue.
- BFS can consume significant memory, especially when the branching factor is high or the tree is deep.
- It guarantees that it explores the shallowest nodes first, but this comes at the cost of increased memory usage.

Depth first Search

- Depth-First Search (DFS) is a strategy for traversing or searching tree or graph structures.
- In the context of artificial intelligence (AI), DFS is often used for exploring possible states in search problems, finding solutions in game playing, planning, and more.

Depth first Search

- Key Concepts
- Traversal Method: DFS explores as far as possible along each branch before backtracking.
- LIFO Structure: Uses a stack (Last-In-First-Out) data structure, either implicitly through recursion or explicitly.
- Completeness: DFS is not guaranteed to find a solution if one exists, especially in infinite search spaces.

Depth first Search

- Example

- From A to F.
- Find out the sequence of nodes to trace the graph, start from F.
- DFS does not consider edge weights, so it is not suitable for finding the shortest path in a weighted graph.
- Edges are given with weights, but DFS will ignore weights and simply follow paths.

```
      2        5
A ------ B ------ D
| \      |      / |
4|  \1   2|    3  |1
|   \    |   /    |
|    \   |  /     |
C ------ E ------ F
      3        1
```

Depth first Search

- Algorithm
- Initialization: Start at the root node (or any arbitrary node in a graph).
- Mark the node as visited.
- Push the node onto the stack.
- Process: While the stack is not empty:
- Pop a node from the stack.
- Process the node (e.g., check if it is the goal).
- For each unvisited adjacent node:
- Mark it as visited.
- Push it onto the stack.

THANK - YOU